Adapting Audio-JEPA for Low-Resource Thai Automatic Speech Recognition

Sawit Koseeyaumporn, Kittipong Tapyou, Warapol Pratumta
Computer Engineering
King Mongkut's University of Technology Thonburi
Bangkok, Thailand

***Abstract*—Thai automatic speech recognition (ASR) is constrained by the limited availability of transcribed speech. Self-supervised speech encoders can reduce this dependence by learning from unlabeled audio, but Audio-JEPA has so far been evaluated mainly as a general-audio representation learner rather than as a transcription system. This paper proposes an adaptation of Audio-JEPA for low-resource Thai ASR. The proposed system replaces coarse spectrogram patches with a high-resolution Thai speech frontend, learns masked latent representations using a context encoder, an exponential-moving-average target encoder, and a predictor, and then fine-tunes the retained encoder with a character-level connectionist temporal classification head. We define controlled experiments using fixed Thai audio pools and 1-, 5-, and 10-hour transcription budgets, with MFCC, Mel-spectrogram, scratch-trained, and multilingual self-supervised baselines. Evaluation reports Thai-normalized character error rate as the primary metric, word error rate with a fixed tokenizer, and separate greedy and beam-search decoding results. The study tests whether Thai speech JEPA pretraining improves label efficiency while making data, compute, and decoding conditions explicit.**

***Index Terms*—automatic speech recognition, Thai speech, self-supervised learning, Audio-JEPA, connectionist temporal classification, low-resource ASR**

I. Introduction

A. Low-Resource ASR Challenge

Automatic speech recognition (ASR) converts speech into text, but supervised systems typically depend on substantial amounts of transcribed audio. Obtaining accurate transcripts requires time and human effort, making this requirement difficult to satisfy when labeled speech is scarce [1]. This study focuses on Thai ASR under a restricted transcription budget, including settings where additional speech must be collected and annotated by the research team. Here, "low-resource" refers primarily to limited labeled training data; the availability of unlabeled audio and computational resources must be considered separately. Recent Thai speech research likewise identifies paired speech–text data scarcity as a constraint [2].

B. From Speech Features to Learned Representations

Conventional ASR systems use acoustic features such as mel-frequency cepstral coefficients (MFCCs) and Mel-spectrograms to describe the speech signal [3], [4]. Self-supervised learning extends this approach by learning representations from audio without requiring transcripts during pretraining. Models such as wav2vec 2.0 and data2vec subsequently adapt these representations to speech recognition using labeled examples [5], [6]. The distinction concerns how representations are learned: a Mel-spectrogram can also serve as the input to a learned encoder. This motivates investigating whether learned features improve recognition when labeled Thai speech is limited.

C. Audio-JEPA

Audio-JEPA learns by predicting representations of masked spectrogram regions from visible context [7]. Its objective operates in an embedding space rather than requiring reconstruction of the acoustic input. This offers a candidate approach for pretraining on unlabeled Thai audio. Whether these representations preserve the information needed for accurate Thai transcription with few labels remains an empirical question.

D. Representation-Based Approach

We therefore propose to investigate Audio-JEPA for low-resource Thai ASR. The study will begin with MFCC and Mel-spectrogram baselines, examine established self-supervised speech encoders, and adapt Audio-JEPA through pretraining or continued pretraining on unlabeled Thai speech. A sequence recognition head will then be trained using limited transcribed Thai data. Comparing these approaches across controlled labeled-data budgets will test whether representation prediction improves recognition beyond conventional features and established self-supervised alternatives. The central research question is whether Audio-JEPA can reduce the labeled-data requirement for Thai ASR under a clearly specified pretraining-data and computation budget.

II. Literature Review

A. Evolution of Speech Representation Learning

Speech representation learning has evolved from fixed acoustic features to models pretrained on unlabeled audio [5], [8], [9]. Self-supervised methods can also be organized according to the targets that they learn to predict [5], [6], [10]–[12].

1) Handcrafted and Statistical Models: Early speech-recognition systems divided audio into short frames and used fixed mathematical transformations to extract salient features [3], [8], [13], [14]. Mel-frequency cepstral coefficients (MFCCs) compressed the signal according to properties of human hearing, producing compact feature representations [3].

Perceptual linear prediction (PLP) incorporated further auditory characteristics, improving robustness across speakers and recording conditions [13]. Researchers also captured temporal changes in speech by computing differences between adjacent features [3], [13]. These features were passed to statistical models: hidden Markov models (HMMs) represented speech timing and sequence [14], whereas Gaussian mixture models estimated acoustic likelihoods [8]. Although computationally efficient, this pipeline separated feature extraction, acoustic modeling, and language modeling, preventing joint optimization [8], [14].

2) Supervised Methods: Supervised methods learn directly from transcribed speech [9], [15]–[19]. Deep neural networks replaced many older statistical components and improved recognition accuracy [9]. Convolutional neural networks captured local acoustic patterns [15], while recurrent neural networks modeled temporal context [16]. Connectionist temporal classification (CTC) enabled training without exact frame-level alignments [17]. Listen, Attend and Spell mapped audio directly to text using attention [18], and the Conformer combined self-attention with convolution to model both global context and local detail [19]. Despite simplifying the recognition pipeline and improving accuracy, these methods still required substantial quantities of labeled audio [9], [18], [19].

3) Self-Supervised Methods: Self-supervised methods learn from raw audio without transcripts by deriving training objectives from the signal itself [5], [6], [10]–[12], [20]–[22]. Contrastive predictive coding (CPC) selected the correct future representation from negative candidates [20], whereas autoregressive predictive coding (APC) predicted future acoustic frames directly [21]. Mockingjay [22] and TERA [10] masked or altered portions of the input and learned to reconstruct them. Later, wav2vec 2.0 applied contrastive prediction to quantized speech representations [5]; HuBERT predicted cluster assignments for masked regions [11]; and data2vec predicted teacher-network representations [6]. These models can pretrain on unlabeled audio and then adapt to downstream tasks with substantially less labeled data [5], [11].

B. Self-Supervised Learning Paradigms

Self-supervised speech-learning methods can be grouped into four categories according to their prediction targets [5], [6], [10]–[12].

1) Reconstruction: Reconstruction methods mask or corrupt parts of an audio input and train the model to recover the missing information [10], [22], [23]. Mockingjay masks acoustic frames and uses a Transformer encoder to reconstruct them [22]. DeCoAR uses bidirectional context to reconstruct acoustic frames [23], while TERA applies alterations along the time, frequency, and magnitude dimensions to improve robustness [10]. This objective is straightforward and requires neither negative sampling nor clustering [10], [22]. However, the model may devote capacity to background noise and low-level acoustic details that are less relevant to speech recognition [10], [23].

2) Contrastive Learning: Contrastive learning trains a model to distinguish the correct latent target from negative candidates [5], [20]. CPC uses the current context to identify the true future representation among distractors [20]. Wav2vec 2.0 masks portions of the latent speech sequence and selects the corresponding quantized representation from distractors sampled from the same utterance [5]. Predicting high-level latent representations can be more useful than reconstructing raw acoustic input [5], [20]. However, performance depends on negative-sampling choices, and the quantization codebook adds training complexity [5].

3) Discrete Unit Prediction: Discrete-unit prediction converts continuous speech into categorical targets and trains the model to predict those targets [11], [12]. HuBERT clusters acoustic features, masks portions of the input, and predicts cluster assignments for the masked regions [11]. WavLM builds on masked speech prediction and augments training with noise and overlapping speech to improve performance under challenging acoustic conditions [12]. Categorical prediction aligns well with speech recognition and avoids explicit negative sampling [11], [12]. Its quality nevertheless depends on the initial clustering or pseudo-labeling process; noisy targets can limit the learned representations [11], [12].

4) Representation Prediction: Representation-prediction methods train a model to predict continuous feature vectors rather than reconstructing raw audio or predicting discrete labels [6], [7], [24], [25]. In data2vec, a student network predicts contextualized representations produced by a teacher network that observes the complete input [6]. Joint-embedding predictive architectures (JEPAs) [24], [25] and Audio-JEPA [7] use a context encoder to predict target-network representations of masked input regions. This approach avoids an explicit input-reconstruction objective, negative-example selection, and externally generated cluster assignments, although downstream usefulness must be evaluated [5], [7], [11].

C. JEPA and Audio-JEPA

1) From Image Representations to Audio: Joint-Embedding Predictive Architecture (JEPA) is a self-supervised learning framework that predicts missing information in latent representation space rather than reconstructing raw input. I-JEPA learns by predicting target image-block embeddings from visible context. Its motivation is to encourage semantic representation learning without the burden of reconstructing every low-level input detail, as required by pixel-reconstruction objectives [25]. For ASR, this motivates learning representations that preserve phonetic, acoustic, and linguistic structure without requiring perfect waveform reconstruction.

A-JEPA extends this principle to audio spectrograms using a context encoder, a momentum-updated target encoder, and a predictor. Its curriculum progresses from random block masking to time–frequency-aware masking. Fei et al. report strong results on audio and speech classification benchmarks, providing evidence that latent prediction can learn useful audio representations [26]. These classification results should be distinguished from continuous-speech transcription. Related

experiments by Riou et al. show that context and target selection materially affect audio representation quality, and that successful image-domain design choices do not necessarily transfer to audio [27].

2) Audio-JEPA and Its Speech Limitations: Audio-JEPA uses Mel-spectrogram patches, random masking, a Vision Transformer encoder, and masked-region latent prediction. Tuncay et al. pretrain on approximately 5,338 hours of AudioSet audio and evaluate frozen representations on X-ARES. They report competitive results against wav2vec 2.0 and data2vec on several tasks with a smaller reported training budget [7]. These comparisons involve different pretraining configurations and do not establish a universal computational advantage.

Performance is mixed: Audio-JEPA is stronger on some general-audio and music tasks and weaker on several tasks requiring fine-grained speech discrimination. Its reported linear-probe evaluation excludes LibriSpeech-100h [7]. Consequently, these results support investigating Audio-JEPA for ASR, but do not establish it as a replacement for speech-specialized encoders such as wav2vec 2.0, HuBERT, or XLS-R. Low-resource Thai transcription remains a downstream evaluation question.

3) Speech-Specific JEPA and Acoustic Grounding: Recent preprints by Ioannides et al. investigate speech-specific grounding. GMM-Anchored JEPA fits a Gaussian mixture model (GMM) on log-Mel features and uses its frozen soft posterior probabilities as auxiliary targets, with their influence decreasing during training. The authors observe representation collapse without grounding in their studied setup. With approximately 50,000 hours of pretraining speech, their anchored model improves ASR, emotion recognition, and slot filling relative to a WavLM-style baseline with matched compute [28]. This is evidence for anchoring under those experimental conditions, rather than proof that all speech JEPA configurations collapse.

The same author group subsequently presents S-JEPA, which predicts soft GMM posteriors through a Kullback–Leibler divergence objective. Training first uses a fixed GMM over MFCCs and then an online GMM over learned encoder features. Under the SUPERB protocol, the authors report the lowest WER among their evaluated SSL systems below 90 million parameters [29]. Unlike Audio-JEPA's continuous embedding regression, this approach explicitly introduces soft clustering targets. The two reports provide related evidence for acoustic grounding, rather than independent demonstrations of unchanged Audio-JEPA.

Taken together, the literature supports JEPA as a viable audio representation-learning direction and motivates testing speech-specific targets for Thai ASR. Our study will examine whether Audio-JEPA representations, with or without acoustic anchoring, retain sufficient temporal and phonetic information for a CTC-based transcription head. This remains a proposed comparison; the cited results do not demonstrate low-resource Thai ASR gains.

D. Low-Resource Training for ASR

1) Self-Supervised Learning with Limited Transcripts: Low-resource ASR is challenging because supervised recognition requires transcribed speech that may be scarce for a target language, dialect, or domain. Mohamed et al. review how self-supervised speech representation learning reduces this dependency by learning from unlabeled speech before adaptation to downstream tasks [30]. This separates the transcription budget from the amount of pretraining audio and the computation required.

Wav2vec 2.0 pretrains on unlabeled waveforms and fine-tunes with CTC. Its large model, pretrained on approximately 53,000 hours of LibriVox audio and decoded with a Transformer language model, reports WERs of 4.8%/8.2% on LibriSpeech test-clean/test-other with ten minutes of labeled data, and 2.9%/5.8% with one hour [5]. These results demonstrate strong label efficiency under large-scale pretraining; they should not be interpreted as training an equally accurate recognizer from only a few minutes of total audio.

HuBERT uses offline clustering to generate hidden-unit targets and learns to predict their assignments at masked positions. Its reported experiments match or improve upon wav2vec 2.0 across fine-tuning subsets of ten minutes, one hour, ten hours, 100 hours, and 960 hours [11]. Together, these studies provide direct ASR evidence for self-supervised pretraining, while their differing model sizes, pretraining corpora, and decoding conditions remain relevant to comparison.

2) Multilingual Transfer: XLS-R scales wav2vec-style pretraining to approximately 436,000 hours across 128 languages. Its evaluation includes BABEL, CommonVoice, Multilingual LibriSpeech, and VoxPopuli, with substantial gains in low- and mid-resource settings [31]. This makes multilingual SSL an important reference for Thai experiments, although its pretraining scale differs substantially from a small project-specific corpus.

The Massively Multilingual Speech (MMS) project further combines self-supervised pretraining with multilingual supervised adaptation. It provides pretrained encoders covering 1,406 languages and an ASR model supporting 1,107 languages. In its reported comparison on 54 FLEURS languages, the multilingual ASR system more than halves Whisper's WER while using a small fraction of its labeled training data [32]. This result supports multilingual label efficiency within that benchmark and comparison, rather than superiority over every Whisper version or deployment setting.

3) Thai-Specific Data and Evaluation Requirements: Thai ASR also requires attention to language-specific structure and transcript consistency. Thai is tonal and lacks explicit word boundaries; alternative written forms for numbers, loanwords, and repetition markers can introduce inconsistencies between training labels and evaluation references. Typhoon ASR Real-time addresses these issues through systematic text normalization and targeted dialect adaptation, including context-dependent number verbalization and repetition-marker handling [33]. Its findings motivate consistent preprocessing

and evaluation for Thai, but are not a controlled test of JEPA or SSL label efficiency.

Thai-specific SSL adaptation is also demonstrated by XLSR-Thai, obtained through continued pretraining on 36,000 hours of Thai speech [2]. Meanwhile, Berrebbi et al. show that domain mismatch can limit self-supervised features and that combining them with conventional spectral features can benefit low-resource recognition [4]. These findings justify retaining MFCC and Mel-spectrogram baselines alongside multilingual and Thai-adapted encoders.

Overall, SSL has strong empirical support for reducing ASR transcription requirements, but its effectiveness depends on data, domain, and training configuration. Our proposed Thai evaluation will compare labeled-data budgets while reporting pretraining hours, model size, initialization, and computation separately. Speaker-disjoint test audio will be excluded from pretraining, and decoding conditions will be controlled where possible. We will report character error rate (CER) and WER with documented text normalization and Thai word segmentation. This design will test whether Audio-JEPA and speech-specific grounding improve Thai recognition under limited supervision.

III. Methodology

A. Research Design

We propose a two-stage recognizer, denoted TAJA-CTC: self-supervised representation learning from Thai speech, followed by supervised character prediction with connectionist temporal classification (CTC) [17]. Figure 1 distinguishes the pretraining graph from the transcription path. This section specifies a proposed implementation; no training outcomes are claimed. The central hypothesis is that masked latent prediction improves label efficiency relative to an identical encoder trained only with CTC.

Audio-JEPA uses an EMA target network and a predictor to learn from masked spectrogram patches [7]. Its reported frontend has 256 time bins per 10-second clip and a patch width of 16 bins. Thus, we estimate only 16 temporal positions per clip, approximately 625 ms apart. Pooling across frequency would retain this coarse timing, while flattening frequency positions would not create additional acoustic time steps. We therefore replace the two-dimensional patch grid with full-frequency, short-duration speech tokens. This is an adaptation of the Audio-JEPA objective and Transformer backbone, rather than a reproduction of its original frontend.

B. Acoustic Frontend and Temporal Encoder

Audio will be converted to mono at 16 kHz. We will compute 80 log-Mel bands using a 400-sample Hann window, 160-sample hop, 512-point FFT, and a 0–8 kHz Mel filterbank. Power values will be floored at 10^{-10} before taking the natural logarithm. Per-band mean and variance will be estimated on training audio only and reused for validation and testing.

Let $X \in \mathbb{R}^{80\times F}$ denote the normalized features. A two-dimensional convolution with kernel and stride $(80, 2)$ and 768 output channels will project each non-overlapping pair of frames into one full-frequency token. Equivalently, a learned affine map projects each 160-dimensional frame pair to 768 dimensions. An odd final frame will be zero-padded, giving $T = \lceil F/2 \rceil$ tokens at a nominal 20 ms stride; batch padding beyond these valid tokens is excluded by length masks. This explicit frequency-to-feature projection preserves temporal order without averaging the utterance. The 25 ms analysis windows overlap locally; masking therefore removes token inputs, not every waveform sample contributing to neighboring windows.

The context encoder contains 12 pre-layer-normalized Transformer blocks, width 768, 12 attention heads, and feed-forward width 3072. These dimensions permit transfer of compatible Audio-JEPA Transformer blocks. One-dimensional sinusoidal positions replace the original two-dimensional positions. There is no classification token or global temporal pooling. A padding mask excludes invalid time steps from self-attention and all losses. The model is bidirectional and intended for offline transcription; streaming latency is outside this experiment.

C. Thai Speech JEPA Pretraining

Write P_θ for the projection stem and E_θ for the context Transformer. A target copy $(P_{\bar\theta}, E_{\bar\theta})$ starts with identical parameters. For each training crop, a temporal mask M will cover approximately 50% of valid tokens using non-overlapping spans of five tokens, corresponding to 100 ms. A final span may be shortened to meet the mask count. The context Transformer receives only visible tokens with their original positions. The target network receives the complete, uncorrupted sequence.

The predictor projects context outputs to width 384, restores masked positions with learned mask tokens, adds their temporal positions, and processes the ordered sequence using six Transformer blocks with 12 heads and feed-forward width 1536. A final linear projection returns 768-dimensional predictions. Let z_t be the target encoder's last-layer output and $\hat z_t$ the predictor output. Our primary objective is normalized latent regression:

$$\mathcal{L}_{\mathrm{J}} = \frac{1}{|M|d} \sum_{t\in M} \left\| \mathrm{LN}(\hat z_t) - \mathrm{sg}[\mathrm{LN}(z_t)] \right\|_2^2, \tag{1}$$

where $d = 768$, sg stops gradients, and LN standardizes each token across channels without learned affine parameters. Normalization and temporal-span masking are proposed speech adaptations; random-token masking is evaluated separately. Only the context branch and predictor receive gradients. After each optimizer update, both the target stem and target Transformer are updated by

$$\bar\theta \leftarrow \tau_s \bar\theta + (1-\tau_s)\theta, \qquad \tau_s = 1-(1-0.996)\frac{1+\cos(\pi s/S)}{2}, \tag{2}$$

where s is the update index and S is the pretraining budget. Target dropout is disabled. Padded tokens are excluded from mask sampling and target statistics.

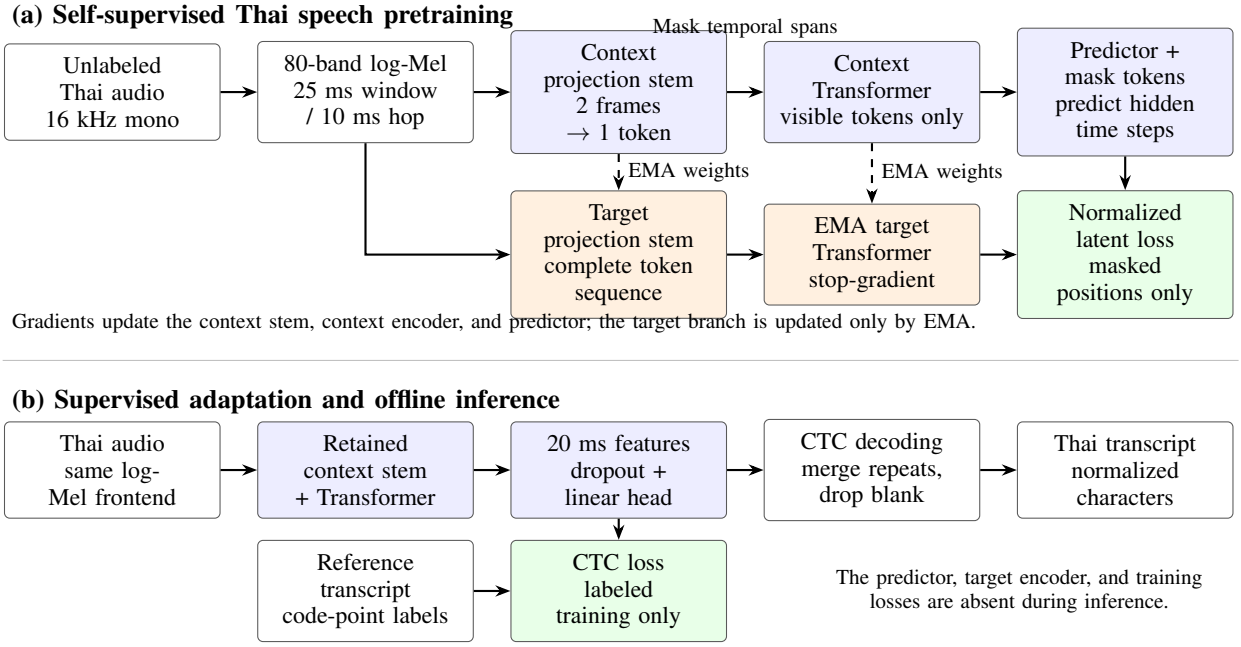


Fig. 1. Proposed Audio-JEPA adaptation for Thai ASR. Solid arrows show data flow; dashed arrows show exponential moving-average (EMA) parameter updates. The full-frequency speech stem produces an ordered 20 ms sequence. Pretraining predicts hidden latent features; supervised CTC adaptation maps those features to text. All architecture and training choices shown are proposed, not measured results.

EMA and normalization do not by themselves establish non-collapse. We will record per-channel standard deviations across unpadded target tokens, effective covariance rank, gradient norms, and held-out JEPA loss. Low regression loss accompanied by near-constant embeddings will be reported as a failed representation-learning run. The predeclared acoustic-grounding ablation below tests an additional stabilization mechanism; it is not silently substituted for the primary objective.

D. Initialization and Checkpoint Transfer

The primary experiment initializes the proposed architecture randomly and pretrains it on the fixed Thai audio pool. A separate transfer experiment will load compatible attention, feed-forward, and layer-normalization weights from an official Audio-JEPA checkpoint, if obtainable from the authors' implementation [34]. The new input stem and temporal positions are not loaded from the original patch embedding. The target is copied from the initialized context network, and the predictor is initialized afresh. A parameter-name and shape audit will record every loaded and skipped tensor. Because the public repository does not currently provide a usable checkpoint link, this transfer comparison is conditional; the from-scratch Thai JEPA experiment does not depend on its availability.

E. Supervised CTC Adaptation and Decoding

After pretraining, the context stem and encoder will be retained; the target branch and predictor will be removed. Unlike the original Audio-JEPA frozen-target evaluation, our downstream choice is the context encoder. A dropout layer with probability 0.1 and a linear classifier produce one distribution over $\mathcal{V} \cup \{\varnothing\}$ per time step, where $\varnothing$ is the CTC blank. For transcript y, training minimizes

$$\mathcal{L}_{\mathrm{CTC}} = -\log \sum_{\pi \in \mathcal{B}^{-1}(y)} \prod_{t=1}^{T} p(\pi_t \mid x). \tag{3}$$

The CTC map $\mathcal{B}$ first merges consecutive identical path labels and then removes blanks [17]. For a target of length U with R adjacent repeated label pairs, we require $T \geq U + R$. Invalid examples will be logged and counted; they will not be concealed by replacing infinite losses with zero. Any training exclusions will be applied to all comparison systems.

Output units are Unicode code points, not word tokens or grapheme clusters. A fixed inventory will include assigned Thai letters, vowels, tone marks and orthographic symbols, ASCII letters and digits, Thai digits, one space token, and an unknown token. Its exact code-point list will be saved before inspecting held-out transcripts. Unsupported symbols map to the unknown token during training; their frequency is reported, and evaluation references retain their normalized characters so that unsupported characters still incur errors.

Text will use Unicode NFC, lowercase Latin letters, and collapsed whitespace. Thai combining vowels and tone marks, numerals, abbreviations, and the repetition mark will be preserved. A versioned list will remove sentence punctuation; no context-dependent number verbalization or repetition expansion is assumed. The same rules apply to labels and decoded text. Primary decoding is greedy CTC. A secondary prefix beam search with beam width 50 and no language model will

isolate search effects. No external text or pretrained language model enters the principal results.

F. Optional Acoustic-Grounding Ablation

Motivated by speech JEPA work using soft clustering anchors [28], we will fit a 100-component diagonal-covariance Gaussian mixture model (GMM) to at most one million normalized training log-Mel frames, sampled with a fixed seed. Adjacent-frame posterior averages provide q_t on the 20 ms grid. An auxiliary linear head on the predictor outputs r_t, and training uses

$$\mathcal{L} = \mathcal{L}_{\mathrm{J}} + \lambda_s \frac{1}{|M|} \sum_{t \in M} D_{\mathrm{KL}}(q_t \| r_t), \tag{4}$$

with $\lambda_s = 0.1 \max(0, 1 - 2s/S)$. The GMM uses no transcripts or held-out audio and stays fixed. This is our specified auxiliary variant, not an exact reproduction of GMM-JEPA or S-JEPA. Its head is discarded before CTC adaptation.

IV. Experimental Settings

A. Corpora, Splits, and Label Budgets

We plan to use Common Voice Scripted Speech 27.0 for Thai, released in September 2026, for in-domain development [35]. The release identifier is cv-corpus-27.0-2026-09-11. Its datasheet lists 33,014 training clips; usable training hours will be measured after filtering rather than inferred from the duration of the full collection. Access, license terms, and the downloaded archive checksum will be recorded before execution.

The published train, development, and test manifests are the starting partitions. We will audit pseudonymous client identifiers and audio hashes, retaining held-out partitions and removing conflicting training records. Records with missing speaker identifiers will not enter the controlled training pool. Remaining development/test speaker conflicts will be resolved by retaining the test partition and removing conflicting development records. We will publish split-generation code and permitted record identifiers, rather than redistribute audio or transcript data; differences from the official splits will be documented.

Training utterances must be decodable, have nonempty normalized transcripts for supervised use, and last 1–20 s. Duration filtering applies only to training; complete held-out utterances will be evaluated with length-aware batches. Exact duplicate audio will be removed, and duplicate transcripts across partitions will be quantified separately. Development and test audio are excluded from local self-supervised training, including when their transcripts would otherwise be hidden.

The target unlabeled budget is 25 h of eligible Common Voice training audio with transcripts withheld. If the audited training split contains less, all eligible audio will be used and the measured duration reported; additional hours will not be assumed. A fixed seed of 2026 selects this pool once for all runs. Within it, nested labeled subsets of 1, 5, and 10 h will be selected by a seeded speaker-balanced ordering of whole utterances. The pool may therefore include labeled-subset audio, but self-supervised training will not read its transcripts. This fixed-pool design separates transcription budget from audio availability. Budgets that cannot be filled after filtering will be reported as unavailable.

Each ordering will first cycle through speakers before selecting further utterances, limiting domination by prolific contributors. For each budget, selection stops before the next utterance exceeds the requested duration and the realized hours are reported. A separate, fixed 1 h development subset will be used for all model selection. Its transcribed hours will be reported in addition to the training-label budget: the "1 h" condition consequently uses approximately 2 h of labeled audio including validation. Three seeds, 13, 37, and 73, determine nested subsets and training randomness; methods share the same subset within each seed.

The Thai `th_th` test split of FLEURS will serve as a second-corpus test [36]. Its training and validation splits will not be used. We will preserve its official test composition, apply the fixed normalization, and report overlap checks where identifiers permit them. Speaker disjointness across independently collected corpora cannot be guaranteed from unavailable metadata. FLEURS measures transfer between read-speech corpora; it does not establish conversational or dialect robustness. Actual utterance, speaker, and duration counts will be tabulated only after the manifests have been audited.

TABLE I
Planned systems. All rows end in a newly trained CTC head. Conditional transfer rows require an obtainable official checkpoint.

System	Initialization and purpose
MFCC-CTC	40 MFCCs plus first/second differences; three bidirectional LSTM layers, 256 units per direction. Conventional-feature baseline.
Mel-CTC	80 log-Mel inputs with the same LSTM stack. Frontend comparison with MFCC-CTC.
Scratch	Proposed 20 ms stem and Transformer; random initialization; supervised CTC only.
Thai JEPA	Same architecture; Thai-only JEPA pretraining, then CTC. Primary proposed system.
Transfer	Audio-JEPA Transformer block initialization; new stem; CTC without Thai JEPA (conditional).
Transfer+Thai	Same transferred initialization followed by Thai JEPA and CTC (conditional).
XLS-R	`facebook/wav2vec2-xls-r-300m`; raw 16 kHz input, new CTC head, same labeled subsets [31], [37].

B. Comparison Systems

Table I defines the main comparisons. All CTC systems use the same labeled subsets, output inventory, validation data, and scoring rules. The scratch model is the essential architectural control: its difference from Thai JEPA estimates the contribution of self-supervised initialization, including its extra pretraining compute. This is not a compute-matched comparison.

The MFCC and Mel baselines use a stride-two temporal projection before the LSTM stack, yielding approximately

TABLE II
PROPOSED OPTIMIZATION SETTINGS FOR THE MAIN EXPERIMENT.

Setting	Planned value
JEPA updates	50,000; save every 5,000; use final checkpoint unless the run fails stability checks
JEPA batch	32 crops of 4 s (128 s nominal audio/update), achieved by accumulation
JEPA optimizer	AdamW, $(\beta_1, \beta_2) = (0.9, 0.95)$, weight decay 0.05, peak learning rate 3×10^{-4}
JEPA schedule	2,000-update linear warmup from 10^{-6}; cosine decay to 10^{-6}
CTC batch	Approximately 120 s unpadded audio per optimizer update
CTC optimizer	AdamW, $(0.9, 0.999)$, weight decay 0.01; encoder 10^{-5}, new head 10^{-3}
CTC schedule	Maximum 10,000 updates; 500-update warmup, then cosine decay; gradient clipping at norm 1.0 in both stages
Fine-tuning	Freeze pretrained stem/encoder for 500 updates while training head; then unfreeze all parameters
Model selection	Development CER every 250 updates; stop after 8 evaluations without improvement, after update 1,000
Regularization	Encoder and predictor dropout 0.1; CTC-head dropout 0.1; no speed perturbation or external noise in the primary experiment

20 ms output spacing. Their frontend statistics are fitted on training audio. XLS-R retains its native waveform frontend. Its multilingual pretraining scale and parameter count differ from the proposed model, so it is a practical reference rather than evidence isolating the JEPA objective. We will report checkpoint provenance and possible upstream evaluation-data exposure separately; excluding local test audio cannot retroactively certify the pretraining data of a public model.

C. Optimization and Compute Protocol

Table II gives proposed starting settings, not optimized or measured results. Pretraining will use randomly selected 4 s crops, padded when shorter; repeated crops do not count as additional unique audio hours. CTC training uses complete utterances and never pairs a truncated waveform with an unmodified full transcript. Duration bucketing and gradient accumulation maintain the effective batch budget.

The scratch and LSTM systems train all layers from the first update with an initial peak rate of 3×10^{-4}; freezing a random encoder is not part of their schedule. For each system family, a pilot on the 5 h training condition will compare multipliers $\{0.5, 1, 2\}$ of its stated learning rates using seed 13. The selected multiplier is fixed across label budgets and all final seeds. Pilot costs count toward reported tuning compute. Conditional transfer runs share the same new-stem initialization within each seed. All completed runs, including instability or CTC alignment failures, will be retained in the experiment log.

The planned hardware target is one CUDA GPU with at least 24 GB memory, using mixed precision and activation checkpointing where necessary. GPU feasibility and runtime have not been measured. Microbatch size will be chosen by a memory smoke test while preserving effective batch duration through accumulation. We will log the actual GPU model, peak allocated memory, precision, optimizer steps, audio processed, elapsed time, and GPU-hours separately for pretraining, tuning, and fine-tuning. PyTorch, audio processing, tokenizer, and decoder versions will be frozen in an environment lockfile before running experiments.

D. Ablations and Interpretation

Ablations will first use the 5 h condition and the same three seeds, changing one factor at a time: (i) 20 versus 40 ms token spacing, implemented with full-frequency two- versus four-frame projections and pretrained separately; (ii) random independent token masks versus five-token spans at the same 50% coverage; (iii) a frozen encoder versus full CTC fine-tuning; and (iv) the primary JEPA objective versus the GMM auxiliary objective. The 40 ms condition will report its CTC-feasibility rate on the common data before model comparison. We will not discard different evaluation utterances for different models.

For temporal-resolution comparisons, the number of optimizer updates and audio seconds per update will be held fixed, while memory and runtime are reported because token counts differ. The frozen-encoder condition trains the CTC head for the full supervised budget. Baseline comparisons answer whether JEPA pretraining is useful at all; the ablations determine whether any improvement depends on time resolution, mask geometry, or acoustic grounding. A falling pretraining loss alone will not be interpreted as an ASR improvement.

E. Evaluation and Reproducibility

Primary evaluation uses corpus-level character error rate (CER). After the shared normalization, whitespace is removed and Unicode code points, including Thai vowel and tone marks, are counted as characters. Secondary word error rate (WER) uses PyThaiNLP's `newmm` word segmentation with whitespace tokens disabled [38]. The package version and dictionary snapshot will be fixed, with no dictionary additions from held-out text. Segmentation is applied identically to references and hypotheses. For either unit type,

$$\mathrm{ER} = 100 \frac{N_{\mathrm{sub}} + N_{\mathrm{del}} + N_{\mathrm{ins}}}{N_{\mathrm{ref}}}. \tag{5}$$

Counts are accumulated over the corpus rather than averaging utterance percentages. Greedy and beam-search results will be shown separately. Tone-mark substitutions/deletions, digit errors, and code-switched utterances will be inspected as descriptive error categories, without treating them as separately optimized test objectives.

For each corpus and label budget, we will report mean and standard deviation over the three runs, plus per-run values. Pairwise CER differences against Scratch and XLS-R will use 2,000 paired bootstrap resamples: speakers where reliable speaker identifiers exist, otherwise utterances with that limitation stated. We will report within-run 95% intervals separately from variation across seeds; three seeds are insufficient to characterize every source of uncertainty. Test sets will be

evaluated only after selecting configurations and checkpoints on development data.

Reproducibility artifacts will include split identifiers and hashes, realized audio/label hours, normalization and vocabulary files, architecture and optimizer configurations, transferred tensor lists, seeds, training curves, checkpoint hashes, decoding commands, and scoring scripts. Learning curves will plot CER/WER against labeled hours; resource tables will distinguish unique pretraining audio from repeatedly processed crops and distinguish trainable parameters from inference parameters. These experiments are intended to test label efficiency, not to claim a completed accuracy gain or universal compute advantage.

REFERENCES

[1] A. Conneau, A. Baevski, R. Collobert, A. Mohamed, and M. Auli, "Unsupervised cross-lingual representation learning for speech recognition," in *Proceedings of Interspeech*, 2021, pp. 2426–2430.

[2] M. Shao, B. Mu, C. Wang, H. Li, Y. Yan, Z. Fu, and L. Xie, "Towards building speech large language models for multitask understanding in low-resource languages," *arXiv preprint arXiv:2509.14804*, 2025. [Online]. Available: https://arxiv.org/abs/2509.14804

[3] S. B. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 28, no. 4, pp. 357–366, 1980.

[4] D. Berrebbi, J. Shi, B. Yan, O. Lopez-Francisco, J. D. Amith, and S. Watanabe, "Combining spectral and self-supervised features for low resource speech recognition and translation," *arXiv preprint arXiv:2204.02470*, 2022. [Online]. Available: https://arxiv.org/abs/2204.02470

[5] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 12 449–12 460.

[6] A. Baevski, W.-N. Hsu, Q. Xu, A. Babu, J. Gu, and M. Auli, "data2vec: A general framework for self-supervised learning in speech, vision and language," in *Proceedings of ICML*, vol. 162, 2022, pp. 1298–1312.

[7] L. Tuncay, E. Labbé, E. Benetos, and T. Pellegrini, "Audio-JEPA: Joint-embedding predictive architecture for audio representation learning," in *Proceedings of IEEE International Conference on Multimedia and Expo (ICME)*, 2025. [Online]. Available: https://arxiv.org/abs/2507.02915

[8] M. J. F. Gales and S. J. Young, "The application of hidden Markov models in speech recognition," *Foundations and Trends in Signal Processing*, vol. 1, no. 3, pp. 195–304, 2008.

[9] G. Hinton *et al.*, "Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups," *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 82–97, 2012.

[10] A. T. Liu, S.-W. Li, and H. yi Lee, "TERA: Self-supervised learning of transformer encoder representation for speech," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 2351–2366, 2021.

[11] W.-N. Hsu *et al.*, "HuBERT: Self-supervised speech representation learning by masked prediction of hidden units," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 3451–3460, 2021.

[12] S. Chen *et al.*, "WavLM: Large-scale self-supervised pre-training for full stack speech processing," *IEEE Journal of Selected Topics in Signal Processing*, vol. 16, no. 6, pp. 1505–1518, 2022.

[13] H. Hermansky, "Perceptual linear predictive analysis of speech," *The Journal of the Acoustical Society of America*, vol. 87, no. 4, pp. 1738–1752, 1990.

[14] L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257–286, 1989.

[15] O. Abdel-Hamid, A. rahman Mohamed, H. Jiang, L. Deng, G. Penn, and D. Yu, "Convolutional neural networks for speech recognition," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 22, no. 10, pp. 1533–1545, 2014.

[16] A. Graves, A. rahman Mohamed, and G. Hinton, "Speech recognition with deep recurrent neural networks," in *Proceedings of IEEE ICASSP*, 2013, pp. 6645–6649.

[17] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," in *Proceedings of ICML*, 2006, pp. 369–376.

[18] W. Chan, N. Jaitly, Q. V. Le, and O. Vinyals, "Listen, attend and spell: A neural network for large vocabulary conversational speech recognition," in *Proceedings of IEEE ICASSP*, 2016, pp. 4960–4964.

[19] A. Gulati *et al.*, "Conformer: Convolution-augmented transformer for speech recognition," in *Proceedings of Interspeech*, 2020, pp. 5036–5040.

[20] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018.

[21] Y.-A. Chung and J. Glass, "Generative pre-training for speech with autoregressive predictive coding," in *Proceedings of IEEE ICASSP*, 2020, pp. 3497–3501.

[22] A. T. Liu, S. wen Yang, P.-H. Chi, P. chun Hsu, and H. yi Lee, "Mockingjay: Unsupervised speech representation learning with deep bidirectional transformer encoders," in *Proceedings of IEEE ICASSP*, 2020, pp. 6419–6423.

[23] S. Ling, Y. Liu, J. Salazar, and K. Kirchhoff, "Deep contextualized acoustic representations for semi-supervised speech recognition," in *Proceedings of IEEE ICASSP*, 2020, pp. 6429–6433.

[24] Y. LeCun, "A path towards autonomous machine intelligence," OpenReview, 2022, version 0.9.2.

[25] M. Assran *et al.*, "Self-supervised learning from images with a joint-embedding predictive architecture," in *Proceedings of IEEE/CVF CVPR*, 2023, pp. 15 619–15 629.

[26] Z. Fei, M. Fan, and J. Huang, "A-JEPA: Joint-embedding predictive architecture can listen," *arXiv preprint arXiv:2311.15830*, 2023, revised January 2024. [Online]. Available: https://arxiv.org/abs/2311.15830

[27] A. Riou, S. Lattner, G. Hadjeres, and G. Peeters, "Investigating design choices in joint-embedding predictive architectures for general audio representation learning," *arXiv preprint arXiv:2405.08679*, 2024. [Online]. Available: https://arxiv.org/abs/2405.08679

[28] G. Ioannides, A. Kieback, J. Goldfeder, L. Pang, A. Chadha, A. Elkins, Y. LeCun, and R. Shwartz-Ziv, "Soft clustering anchors for self-supervised speech representation learning in joint embedding prediction architectures," *arXiv preprint arXiv:2602.09040*, 2026. [Online]. Available: https://arxiv.org/abs/2602.09040

[29] ——, "S-JEPA: Soft clustering anchors for self-supervised speech representation learning," *arXiv preprint arXiv:2606.19398*, 2026. [Online]. Available: https://arxiv.org/abs/2606.19398

[30] A. Mohamed, H. yi Lee, L. Borgholt, J. D. Havtorn, J. Edin, C. Igel, K. Kirchhoff, S.-W. Li, K. Livescu, L. Maaløe, T. N. Sainath, and S. Watanabe, "Self-supervised speech representation learning: A review," *IEEE Journal of Selected Topics in Signal Processing*, 2022.

[31] A. Babu, C. Wang, A. Tjandra, K. Lakhotia, Q. Xu, N. Goyal, K. Singh, P. von Platen, Y. Saraf, J. Pino, A. Baevski, A. Conneau, and M. Auli, "XLS-R: Self-supervised cross-lingual speech representation learning at scale," in *Proceedings of Interspeech*, 2022, pp. 2278–2282.

[32] V. Pratap, A. Tjandra, B. Shi, P. Tomasello, A. Babu, S. Kundu, A. Elkahky, Z. Ni, A. Vyas, M. Fazel-Zarandi, A. Baevski, Y. Adi, X. Zhang, W.-N. Hsu, A. Conneau, and M. Auli, "Scaling speech technology to 1,000+ languages," *arXiv preprint arXiv:2305.13516*, 2023. [Online]. Available: https://arxiv.org/abs/2305.13516

[33] W. Sirichotedumrong, A. Na-Thalang, P. Manakul, P. Taveekitworachai, S. Sripaisarnmongkol, and K. Pipatanakul, "Typhoon ASR real-time: FastConformer-Transducer for Thai automatic speech recognition," *arXiv preprint arXiv:2601.13044*, 2026. [Online]. Available: https://arxiv.org/abs/2601.13044

[34] L. Tuncay, "Audio-JEPA: Official implementation," GitHub repository, accessed September 22, 2026. [Online]. Available: https://github.com/LudovicTuncay/Audio-JEPA

[35] Mozilla Foundation, "Common voice scripted speech 27.0—thai," Mozilla Data Collective, 2026, release cv-corpus-27.0-2026-09-11; accessed September 22, 2026. [Online]. Available: https://mozilladatacollective.com/datasets/cmu5x5tj000f2nq0755h8qcy5

[36] A. Conneau, M. Ma, S. Khanuja, Y. Zhang, V. Axelrod, S. Dalmia, J. Riesa, C. Rivera, and A. Bapna, "FLEURS: Few-shot learning evaluation of universal representations of speech," *arXiv preprint arXiv:2205.12446*, 2022. [Online]. Available: https://arxiv.org/abs/2205.12446

[37] Meta AI, “XLS-R-300m model card,” Hugging Face, accessed September 22, 2026. [Online]. Available: https://huggingface.co/facebook/wav2vec2-xls-r-300m

[38] PyThaiNLP Contributors, “PyThaiNLP tokenization api,” Software documentation, describes newmm and whitespace handling; experiment software version will be pinned separately. [Online]. Available: https://pythainlp.org/docs/2.0/api/tokenize.html